An Empirically Grounded Reference Architecture for Software Supply Chain Metadata Management

Nguyen Khoi Tran
CREST
The University of Adelaide
Adelaide, Australia
nguyen.tran@adelaide.edu.au

Samodha Pallewatta
CREST
The University of Adelaide
Adelaide, Australia
samodha.pallewatta@adelaide.edu.au

M. Ali Babar
CREST, The University of Adelaide
Cyber Security Cooperative Research Centre
Adelaide, Australia
ali.babar@adelaide.edu.au

ABSTRACT

With the rapid rise in Software Supply Chain (SSC) attacks, organisations need thorough and trustworthy visibility over the entire SSC of their software inventory to detect risks early and identify compromised assets rapidly in the event of an SSC attack. One way to achieve such visibility is through SSC metadata, machine-readable and authenticated documents describing an artefact's lifecycle. Adopting SSC metadata requires organisations to procure or develop a Software Supply Chain Metadata Management system (SCM2), a suite of software tools for performing life cycle activities of SSC metadata documents such as creation, signing, distribution, and consumption. Selecting or developing an SCM2 is challenging due to the lack of a comprehensive domain model and architectural blueprint to aid practitioners in navigating the vast design space of SSC metadata terminologies, frameworks, and solutions. This paper addresses the above-mentioned challenge by presenting an empirically grounded Reference Architecture (RA) comprising of a domain model and an architectural blueprint for SCM2 systems. Our proposed RA is constructed systematically on an empirical foundation built with industry-driven and peer-reviewed SSC security frameworks. Our theoretical evaluation, which consists of an architectural mapping of five prominent SSC security tools on the RA, ensures its validity and applicability, thus affirming the proposed RA as an effective framework for analysing existing SCM2 solutions and guiding the engineering of new SCM2 systems.

CCS CONCEPTS

• **Software and its engineering** → *Software design engineering*; • **Security and privacy** → **Software and application security**.

KEYWORDS

Software Supply Chain, SSC Metadata, SBOM, Software Provenance, Reference Architecture, Systematisation of Knowledge, Empirically Grounded

ACM Reference Format:
Nguyen Khoi Tran, Samodha Pallewatta, and M. Ali Babar. 2024. An Empirically Grounded Reference Architecture for Software Supply Chain Metadata Management . In *28th International Conference on Evaluation and Assessment in Software Engineering (EASE 2024), June 18–21, 2024, Salerno, Italy.* ACM, New York, NY, USA, **??** pages. https://doi.org/10.1145/3661167.3661212

EASE 2024, June 18–21, 2024, Salerno, Italy

ACM ISBN 979-8-4007-1701-7/24/06.
https://doi.org/10.1145/3661167.3661212

1 INTRODUCTION

In recent years, a meteoric rise in software supply chain (SSC) attacks has occurred. For instance, the 8th Annual State of the Software Supply Chain Report by Sonatype in 2023 [**?**] reported a 742% annual increase in SSC attacks in the past three years. An SSC attack combines an upstream attack, where malicious codes are injected into a software artefact via a compromised life cycle activity, and a downstream attack on the consumers who use the artefact [**?**]. Lack of oversight in an artefact's lifecycle makes upstream attacks possible, while a lack of visibility over the distribution and consumption of artefacts opens the door for downstream attacks. Therefore, securing an inventory of software assets requires a thorough and trustworthy visibility over its entire SSC. Such visibility can be provided by *SSC metadata* - the information about an artefact's life cycle, such as how it was constructed and the "ingredients" utilised in its construction [**?**].

The utility of SSC metadata in identifying and mitigating SSC attacks depends on the ability of software security tools to authenticate and interpret this information. Therefore, *a new class of formal SSC metadata that is machine-readable and authenticated has emerged*[**?**]. This paper focuses on this SSC metadata class which includes various metadata types such as *Software Bill of Material (SBOM)*, a formal and machine-readable list of ingredients that make up software components [**?**], *provenance* statements that describe how and by whom a software artefact was produced [**?**] and *software attestation*, which are digitally signed statements about a software artefact [**?**]. They are constructed based on a predefined standard (e.g., Software Package Data Exchange (SPDX),[1] CycloneDX [2], and in-toto attestation framework[3]), digitally signed and, optionally, notarised (e.g., Supply Chain Integrity, Transparency and Trust (SCITT) [**?**]) to ensure their authenticity and verifiability.

Adopting SSC metadata requires practitioners to use software tools to generate, sign, distribute, and consume this information. We denote this software suite as a **S**oftware Supply **C**hain **M**etadata **M**anagement system **(SCM2)**. Practitioners face two main challenges when assembling or developing an SCM2: (1) the diversity and overlapping frameworks and terminology related to SSC metadata and (2) the diversity of software solutions making up the design space of an SCM2. As we described above, SSC metadata standards and frameworks vary not only in syntax but also in semantics and

[1] https://spdx.dev/ [2] https://cyclonedx.org/ [3] https://in-toto.io/

content, which are tied to how they define an SSC and the aspects of an SSC they aim to describe (e.g., whether an SSC is a sequence of steps performed by different entities to construct a software artefact [**?**] or a network of providers and consumers of software artefacts [**?**]. These diverse standards and frameworks are actualised by their own toolchains. Moreover, various open-source tools (e.g., Syft[4]) and proprietary platforms have emerged to address specific aspects of SSC metadata's life cycle, such as generating and signing SSC metadata documents. These tools differ regarding their supported SSC metadata types and standards. They also vary in capability and correctness. The accuracy and completeness of the generated SBOMs caused by tooling competence have been a shared concern among practitioners, according to a recent empirical study about SBOM adoption [**?**].

The described challenges highlighted the need for a Systematisation of Knowledge (SoK) of SSC metadata to aid practitioners in assembling and developing SCM2 systems. Such a SoK could offer a much-needed comprehensive domain model regarding SSC metadata to facilitate clear communication between architects and stakeholders to identify and define the organisation's information needs regarding SSC and select the suitable types of SSC metadata. An SoK could also provide an architectural blueprint of SCM2 systems, specifying the system's context, scope, and functional decomposition to aid architects in identifying, assessing, and selecting platforms and tools to construct an SCM2. Even though SSC metadata appears in most existing SSC security frameworks (e.g., SLSA [**?**], SCVS [**?**], SSF [**?**]), a comprehensive domain model and architectural blueprint of SSC metadata and SCM2 have not been established.

To this end, this paper presents such an SoK in the form of an ***Empirically Grounded Reference Architecture (RA)*** for SCM2. An RA can be considered a template for software systems, providing a common vocabulary, taxonomy, architectural vision, and building blocks to guide the development of future systems [**?**]. RAs have traditionally been constructed to provide SoK and guidance for Service-oriented computing [**?**], Cloud computing [**?**], Big data [**?**], and Internet of Things [**?**], where a diverse and decentralised set of stakeholders are required to interoperate and cooperate. Today's SSC metadata management exhibits similar multiplicity due to the diversity of frameworks, standards, tool sets, and stakeholders, making an empirically grounded RA a suitable solution.

Systematically designing RAs through an empirically grounded approach facilitates capturing the key concepts and elements of the state-of-the-art frameworks and architectures, thus improving the validity and reusability of the RA [**? ?**]. Hence, we follow the empirically grounded RA design methodology proposed by Galster and Avgeriou [**?**], which includes two aspects for the systematic design of RAs: use of an empirical foundation where practice-proven concepts and building blocks are used for constructing an RA and validity of the constructed RA where the proposed RA is evaluated in terms of its correctness and applicability. As SSC metadata management is a novel and compelling area of interest among software practitioners [**?**], we use state-of-the-art industry-driven and peer-reviewed SSC security frameworks and architectures to establish a solid empirical foundation for the RA. Our proposed RA for SCM2 comprises two components: (1) the *domain model* (Section **??**) presents the key concepts related to SSC metadata and the relationships between them, accompanied by the life cycle of SSC metadata and the related actors. The *architectural blueprint* describes the scope and positioning of SCM2 within an SSC (Section **??**) and the function units making up an SCM2 instance (Section **??**). To ensure the validity of the proposed RA, we evaluate its correctness and utility by mapping the SCM2 elements of five prominent SSC security tools onto the concepts and architecture of the RA. Based on the architectural mapping, we discussed the state of practice of SCM2 and presented a reference instantiation of SCM2 from the existing open-source tools.

[4] https://github.com/anchore/syft

2 METHODOLOGY

Our methodology was developed based on the systematic design approach proposed by Galster and Avgeriou [**?**] and comprised six steps (see Figure **??**):

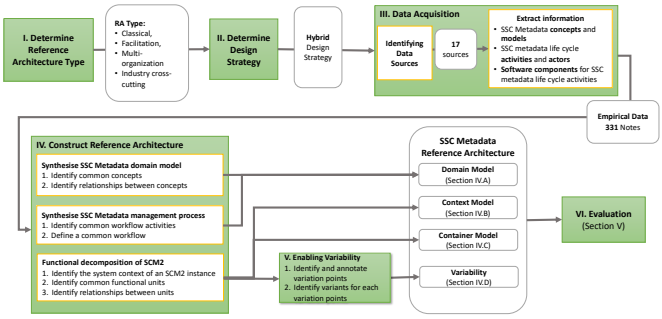

Figure 1: Methodology for constructing the reference architecture for SSC Metadata life cycle management

Step 1 - Determine RA type: This crucial decision determines the design strategy and data collection method for constructing an RA. Based on Angelov's classification system [**?**], we classified our RA as a *classical, facilitation* RA intended for use by *multiple organizations* in an *industry cross-cutting context*. This means that our RA would be based on experiences from preexisting systems, with the primary goal of facilitating the design and development of new systems. It would be used by multiple organizations, both within and outside the software industry, that require access to SSC metadata.

Step 2 - Determine Design Strategy: A *hybrid design strategy* was adopted, meaning both industry-driven SSC security frameworks and peer-reviewed academic articles were utilised as inputs for the construction of the proposed SCM2 RA.

Step 3 - Data Acquisition: The first activity in this step was *identifying the data sources for constructing the empirically grounded RA*. A preliminary set of SSC security frameworks and architectures were identified based on the authors' experiences with and discussions within SSC security industry projects. This preliminary selection was expanded using the snowballing sampling technique [**?**]. If a framework had multiple revisions, only the latest version was selected. This selection process resulted in 13 industry-driven reports and standards and four peer-reviewed articles which, together, specify **eleven SSC security frameworks** that served as the empirically grounded data input for our RA. Table **??** presents

an overview of the chosen frameworks, focusing on their coverage of SSC metadata and its life cycle. We classify the coverage into four classes: "0" denotes that a framework does not offer any details about an SSC metadata life cycle stage, "1" indicates that it provides an overall description, "2" indicates that it describes workflow activities and actors, and "3" denotes that it provides functional decomposition and describes involved software components. For brevity, we make the details of these frameworks available in the online appendix of the paper (see Section **??**).

The second activity was *extracting information* from data sources using the document analysis method [**?**]. The authors independently analysed the sources and extracted relevant information regarding SSC metadata, SSC metadata life cycle activities and actors, and software components introduced to carry out SSC metadata life cycle activities. The extracted information were captured in **331 digital note cards** and stored in a version-controlled repository. The authors incrementally review the note cards and organised them into a knowledge graph in weekly discussion sessions. We leveraged Obsidian[5], a personal knowledge base system, as the tool to capture digital notes and construct the knowledge graph. The resulting knowledge graph can be found in the online appendix of the paper (see Section **??**).

Step 4 - Reference Architecture Construction: The RA consists of a domain model and an architectural blueprint. The domain model comprised (1) *a static model* showing concepts related to SSC metadata and how they relate with each other and (2) *a dynamic model* that describes the life cycle of SSC metadata. These models were derived from the common concepts and activities captured in the knowledge graph.

The architectural blueprint for SCM2 was constructed from the *union* of architectural components captured in the knowledge graph. Union rather than intersection was chosen because this approach extends the coverage of the constructed RA. This design was driven by our observation that none of the existing frameworks provides a complete coverage across SSC metadata types and life cycle activities. The constructed RA was documented using two higher-level abstractions defined by the *C4 architectural view model* [**?**]. Particularly, we utilised the system context model to describe how an SCM2 instance fits in to the overall IT environment and the container model to describe the functional units within an SCM2 instance.

Step 5 - Enabling Variability: Instantiating software architecture from an RA necessarily introduces variations such as the modification or removal of functional units. An RA can support variability by specifying the variation points (architectural elements where changes might occur) and variants (the possible options for those variation points) [**?**]. We identified the variation points by searching for SSC metadata life cycle activities and functional units where the analysed frameworks and architectures diverged. The architectural components and activities utilised by these frameworks and architectures represent the variants. We used textual annotation to describe the variability of the proposed RA.

[5] https://obsidian.md

Step 6 - Evaluation: To evaluate the correctness and usefulness of the empirically grounded RA, we follow the architectural mapping-oriented RA evaluation approach proposed in [**?**] and [**?**]. The evaluation involves mapping prominent SSC security solutions to the proposed RA and analysing how they comply. We selected 5 security tools available for the use by practitioners and widely recognized for management of SSC metadata. We will discuss detailed evaluation methodology in Section **??**.

3 SCM2 REFERENCE ARCHITECTURE

3.1 Domain Model

3.1.1 SSC metadata Concepts. We organised the concepts relevant to SSC metadata and the relationship between them into four clusters (Figure **??**), representing four perspectives to define SSC metadata: the *SSC activities* that create and consume metadata, the *SSC artefact* that metadata describes, the *SSC metadata actors* who act upon the metadata, and finally the taxonomic relationships among SSC metadata types themselves.

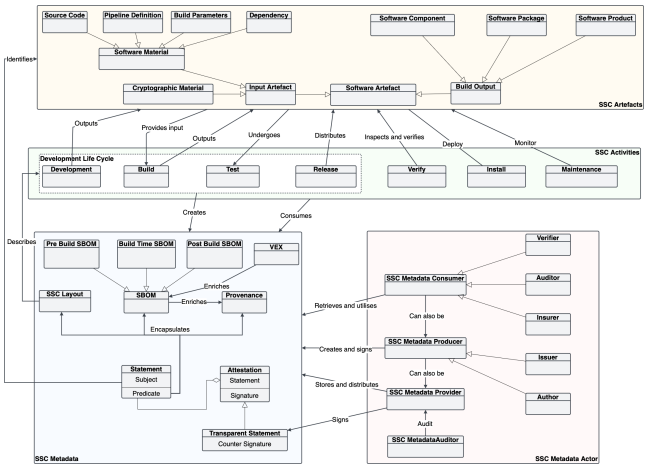

Figure 2: Domain Model for SSC and SSC Metadata Concepts

SSC Activities: An SSC can be considered a series of coordinated *SSC activities* performed by a network of actors to create and distribute software products to end-users [**? ? ? ? ?**]. On the producer side, SSC activities include *development*, *build*, *test*, and *release*. The activities on the consumer side after retrieving a software product include *verification* (e.g., checking digests and verifying the product against its metadata), *installation*, and *maintaining* the software product throughout its life span.

SSC artefacts: SSC activities consumes and produces *software artefacts*. The existing SSC security frameworks defined artefacts in various ways, from any "immutable blob of data" [**?**] to "item that is moving along the supply chain" [**?**] to the principal output of a secure software factory that downstream users would consume [**?**]. Our domain model classifies SSC artefacts into input and output artefacts. SSC activities consume input artefacts include *software materials* [**?**] such as *source code*, build *pipeline definition*, external *build parameters* [**?**] and *dependency*, which are other artefacts that need to be collected before building a software [**?**]. The input artefacts also include cryptographic material such as certificates,

Table 1: SCM2 Support of existing SSC Security Frameworks

Framework/ Architecture	Scope	Supported SSC Metadata Types	Supported SSC Metadata Standards	Metadata Life Cycle Coverage		
				Generation	Sharing	Consumption
SEI SCRM [? ? ?]	SSC Concepts	N/A	N/A	N/A	N/A	N/A
In-toto Attestation Framework [?]	SSC Metadata	Attestation Provenance (Partially)	In-toto Attestation Specification	3	1	3
Software Delivery Governance [?]	SSC Metadata	Provenance	N/A	1	3	1
SCVS [?]	SSC Metadata	SBOM	SPDX	1	0	1
SBOM Generation Playbook [? ? ? ?]	SSC Metadata	SBOM	Any	2	1	1
SBOM Sharing Playbook [? ?]	SSC Metadata	SBOM	Any	0	2	1
SBOM Consumption Playbook [?]	SSC Metadata	SBOM	Any	0	1	2
SBOM Tool Classification [?]	SSC Metadata	SBOM	Any	1	0	1
SSF Architecture [?]	SSC Metadata	Attestation	Any	3	3	0
SCITT [?]	SSC Metadata	Attestation	SCITT Transparent Statement	1	3	2
SLSA [?]	SSC Metadata	Provenance Attestation	SLSA Provenance Model, SLSA Software Attestation Model	1	1	1

tokens, and signing keys [?]. The output artefact, also known as *build output*, contains the output produced by a build process [?]. *Software components*, the focus of SBOM-focused reports by NTIA [?] and Core Infrastructure Initiative [?], can be considered fine-grained build outputs that another software can call. *Software packages*, the focus of SLSA framework [?], can be a collection of software components published for others. A *software product* [?] can be considered a synonym or a super-set of software packages.

SSC metadata provides machine-readable and authenticated descriptions of SSC artefacts and activities. At the core of an SSC metadata document is a collection of *statements*, consisting of a set of *predicates* about a *subject* such as an SSC activity or artefact [?]. The predicates can describe various types of information:

- *Software Bill of Material (SBOM)* is a machine-readable nested inventory of components within a package or product [?]. SBOM can be written in interoperable formats such as SPDX, CycloneDX, and SWID. SBOM can be further enriched by *Vulnerability Exploitability eXchange (VEX)* documents, which integrate vulnerability information with component data from SBOM [? ? ?].
- *Provenance* describes how an artefact was produced. It can be considered a claim that some entity produced one or more artefacts by executing a build pipeline definition according to some parameters, possibly using other artefacts as input [?]. An alternative definition by the SCVS framework [?] considers provenance as the origin and chain of custody of a software component.
- *SSC Layout* is a document providing an ordered list of steps, requirements for the steps, and actors responsible for each step that is required to be carried out in the SSC to create a software product [?].

Software attestation is generated when a digital signature authenticates the statements. By signing an attestation, the authors of an SSC metadata document claim that the predicates (e.g., SBOM, provenance) about a subject (e.g., a software package) are factual [?]. A trusted authority can countersign an attestation to prove its existence and validity. A countersigned attestation can be denoted as a *transparent statement* [?].

SSC Metadata Actors carry out life cycle activities of SSC metadata documents. They can be organised into three classes.

- *SSC Metadata Producers* create and sign SSC metadata. Metadata producers can be project owners [?] or package producers [?], who are responsible for defining the layout of the supply chain of a software artefact. Metadata producers can also be functionaries (e.g., developers, build systems, etc.) that handle individual steps in the life cycle of a software artefact [?]. SSC metadata producers are also known as *issuer* [?] and *author* [? ?].
- *SSC Metadata Consumers* retrieve and utilise SSC metadata [? ? ?]. A *verifier* is a consumer who leverages SSC metadata to verify the authenticity and integrity of software artefacts [? ? ?]. The inspection process of the artefacts also creates more metadata, such as VEX. Other types of consumers include *insurers* and *auditors* [?] who leverage SSC metadata to assess an organisation for regulatory or insurance purposes.
- *SSC Metadata Providers* [?] operate the intermediary services that bridge SSC metadata producers and consumers. Providers can also act as a notary, vouching for the existence and the correctness of SSC metadata documents registered by producers [?]. A *SSC metadata auditor* is a particular type of auditor who assesses the trustworthiness of the providers based on the log or provenance of the SSC metadata documents distributed by providers [?].

3.1.2 SSC Metadata Life Cycle. The life cycle of SSC metadata comprises ten activities that can be organised into generation, sharing, and consumption phases. Figure ?? depicts the life cycle activities and the actors handling those activities. Where the analysed frameworks and architectures diverge, we present their alternative activities as variations.

SSC Metadata Generation Phase consists of 3 main activities starting with *SSC metadata creation* by individuals or automated tools. The supplier playbook by NTIA [?] defines a three-step process that includes identifying software components for delivery, acquiring their data, and capturing the data in an SBOM format. NTIA [?] further prescribes that SBOM authors must include SBOMs of all the dependencies or create those SBOMs on a best-effort basis should such information be unavailable. The in-toto attestation framework [?] prescribes that all "functionaries", actors participating in the creation of software artefacts, must record metadata after finishing their activity. The SLSA 1.0RC specification [?] also requires that build processes automatically generate provenance attestation describing how the artefact was built (by whom, using which process or command, with which input artefacts). Similarly, the Secure Software Factory framework [?] also prescribes automated metadata generation carried out by a software component embedded into build pipelines.

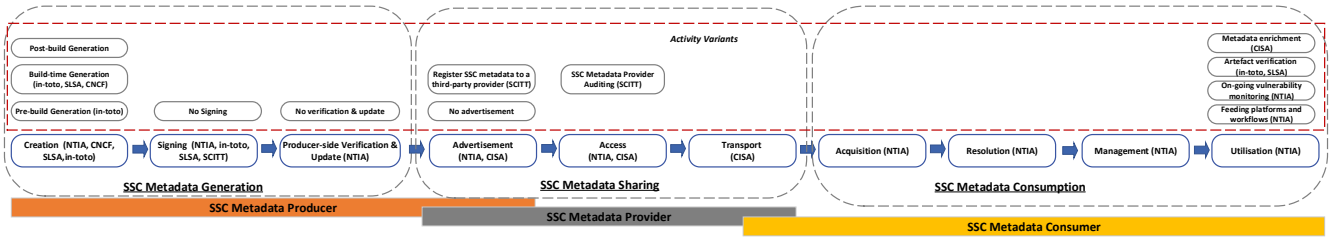

Figure 3: Life cycle of SSC Metadata

There are three variants of carrying out the SSC metadata creation: *pre-build* generation, *build-time* generation, and *post-build* generation [**?**]. Pre-build generation activity creates metadata about source code, such as static dependency declaration in the form of `requirements.txt` in Python projects and `package.json` in NodeJS projects. The SSC layout introduced by the in-toto framework [**?**] is also a product of a pre-build generation activity. The build-time generation activity generates SSC metadata during or before the end of the build process. It is the most common variant of SSC metadata creation activity, featured in most frameworks, including in-toto [**?**], SLSA 1.0RC [**?**], and Secure Software Factory [**?**]. The final variant is the post-build generation, which captures metadata such as VEX from automated tests and auditing activities performed on the newly built artefacts [**?**].

The second activity in the generation phase is *signing* the generated SSC metadata [**?**]. This activity is required by the in-toto framework [**?**] and the SCITT architecture [**?**]. The second and third levels of the build track of SLSA 1.0RC specification [**?**] also require SSC metadata to be digitally signed. It should be noted that signing SSC metadata is not mandatory. For instance, the supplier handbook [**?**] and level one of the build track of SLSA 1.0RC consider metadata signing an optional activity. Therefore, we introduce *"no signing"* as a variant of the signing activity.

The third activity is *verifying and updating* the content of the created metadata. This activity was introduced in the NTIA's supplier playbook [**?**] as a remedy for potential errors due to the SBOM creation software tools and errors inherited from upstream SBOMs. Interestingly, we could not find the provider-side verification and metadata update in other frameworks and architectures. Therefore, we introduce *"no verification and update"* as a variant.

SSC Metadata Sharing Phase bridges the SSC metadata producers and consumers [**? ?**]. The first activity in this phase is *"advertisement"*, which SSC metadata producers perform to inform consumers of the availability of SSC metadata documents and how to access them [**?**]. Producers also use the advertisement to notify consumers of corrections or updates of a previously published SSC metadata document. Producers can handle the advertisement themselves or *register SSC metadata documents with a third party provider* for distribution [**? ?**]. Alternatively, producers can *omit the advertisement*. In this case, consumers would query producers for SSC metadata of the received software artefacts [**?**].

The second activity is *access*, in which consumers' requests for SSC metadata are authorised according to some policies specified by the producer. These policies are defined in a fine-grained manner, limiting access to specific versions or potions of SSC metadata documents [**?**]. The responsibility for assessing and enforcing access policies lies with either producers or providers who host the SSC metadata documents. If a third-party provider is utilised, the *provider auditing* activity could be performed [**?**].

The metadata sharing phase concludes with *transport* SSC metadata documents from a producer or a provider to the authorised consumers using a secure mechanism [**? ?**]. According to [**?**], transfer can be done from single point to single point or single point to multi point based on the targeted consumers.

SSC Metadata Consumption Phase comprises four activities [**?**]. The first activity is the *acquisition* of SSC metadata documents via the sharing mechanisms provided by producers or providers. The second activity is *resolution*, in which a consumer maps subjects of the statements in an SSC metadata document onto software artefacts within their inventory to establish a link between them. This step might also involve resolving transitive dependency identified in an SBOM.

The third activity is applying *content management* technique on the acquired SSC metadata documents, such as defining and enforcing policies about their storage, data retention and life cycle. The consumption phase ends here if the SSC metadata documents are not utilised in any enterprise and IT workflow. Otherwise, the last activity is *utilisation* where the acquired SSC metadata documents are *fed into platforms or workflows* (e.g., Application Security Posture Management, Software Asset Management), utilised for *on-going vulnerability monitoring*, employed for in artefact verification processes [**? ?**]. An organisation can also *enrich* an acquired SSC metadata document, such as by appending their own VEX, and share it with other consumers [**?**].

3.2 Architectural Blueprint

This section presents the architectural blueprint for the SSC Metadata Management System (SCM2), which is a suite of software tools for SSC metadata actors (defined in section **??**) to carry out the life cycle activities (defined in section **??**). We construct the architectural blueprint based on the extracted empirical data and present the architectural blueprint using two models: Context Model and Container Model.

3.2.1 Context Model. The system context diagram in Figure **??** presents the positioning and scope of SCM2, reflected via its interactions with three types of actors and eight types of external systems.

The actors and external systems interacting with SCM2 can be organised into two groups, namely producer and consumer. The

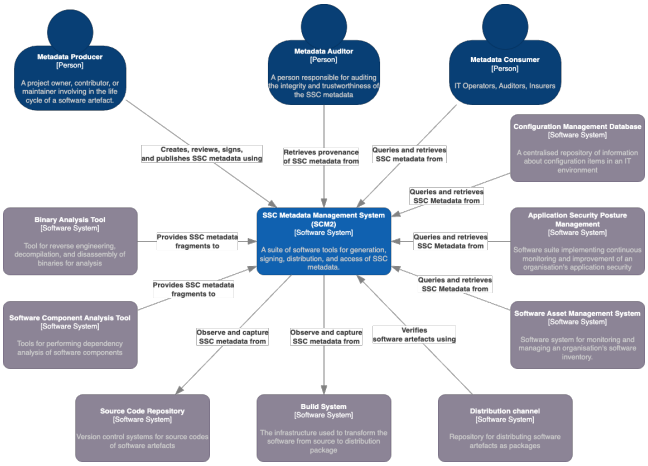

Figure 4: Context model of SCM2

producer group contains the metadata producers, binary analysis tools, software component analysis tools (SCA), source code repositories, and build systems. From the perspective of SCM2, the first three actors and external systems are push-based producers because they push fragments of SSC metadata to SCM2 for constructing SSC metadata documents. On the other hand, source code repositories and build systems are pull-based producers because SCM2 observes and pulls metadata fragments from these systems when constructing SSC metadata document for an artefact.

The SSC metadata generation process of SCM2 can be initiated in two ways. Firstly, it can be triggered by the build process as a part of a CI/CD pipeline. This workflow is suggested by many frameworks [**?** **?** **?**] and particularly relevant to the scenarios where the SSC metadata producer of a software artefact is also the developer and project owner of an artefact. The second approach is to have the process triggered manually by SSC metadata producer, when the need for SSC metadata arises, such as to document their software inventory with SBOM documents. This workflow is particularly relevant to organisations that primarily consume rather than produce software artefacts. The producers might rely on SCA and binary analysis tools to extract the necessary information about their software artefacts. Due to the separation of concern, we decided to place SCA and binary analysis tools outside the scope of SCM2. It means that SCM2 does not contain the ability to perform composition analysis or reverse engineering binaries. It only contains the ability to generate SSC metadata document from the output of these tools.

The second group of actors and external systems of SCM2 consists of metadata consumers, configuration management databases, application security posture management systems, software asset management systems, distribution channels, and metadata auditors. All actors and external systems in this group besides auditors query and retrieve SSC metadata documents from SCM2 for various purposes, such as verifying an incoming software artefact before installation. The metadata auditor, on the other hand, consumes the provenance of the SSC metadata documents in order to assess the trustworthiness of the SSC metadata and the SCM2 itself.

3.2.2 Container Model. The container diagram in Figure **??** presents the distribution of the functionality of an SCM2 system across 13 containers. It should be noted that a container in the C4 architectural view model denotes a separately runnable unit that executes code or stores data rather than a software container (e.g., Docker). We defer the binding of containers to specific technologies to the design time when architects instantiate concrete SCM2 architecture from the RA. The containers of an SCM2 system organised into five groups according to the SSC metadata life cycle phase in which they manage.

SSC metadata generation group contains three containers. *SSC Metadata Editor* provides a graphical interface for reviewing and editing SSC metadata document. This tool belongs to the type "Edit" of the category "Produce" in the tool classification taxonomy by NTIA [**?**]. It supports SSC metadata producers in authoring pre-build metadata (e.g., in-toto's SSC layout metadata [**?**]), supplying post-build metadata and verifying the auto-generated SSC metadata documents [**?**].

Pipeline Observer captures information about tasks in a build pipeline, such as fetching source code and dependencies, building and testing artefacts. The captured evidence is the input for generating SSC metadata documents. Pipeline observer was introduced in the Secure Software Factory reference architecture [**?**].

SSC Metadata Generator creates SSC metadata document from the input information according to a prescribed standard such as SPDX [**?**]. It should be noted that most existing tools combine the metadata generator with a Software Composition Analysis tool or a pipeline observer (e.g., by integrating the generator tool into a build pipeline). In this reference architecture, we model the metadata generator as a separate function unit to follow the separation of concern principle and facilitate the reuse of the software logic for generating SSC metadata document for different standards and templates.

SSC metadata signing group consists of one container: *SSC Metadata Signer*. The signer provides metadata producers with the ability to sign the generated SSC metadata documents, effectively turning them into authenticated statements about a software artefact (software attestations). For example, the "cosign" component[6] of the "sigstore" project can be used as an SSC metadata signer by utilising its built-in support for the in-toto attestation framework [**?**].

SSC metadata publishing group comprises two containers. *SSC Metadata Publisher* provides metadata producers with the functionality for publishing and advertising the generated SSC metadata documents. This functionality reflects and implements the "advertisement" phase described in the SBOM sharing life cycle [**?** **?**]. The publisher can also implement the notarising workflow described by the SCITT architecture [**?**] where an external authority verifies and countersigns a submitted SSC metadata document to attest for its trustworthiness. The client-side of sigstore's rekor[7] can be utilised to implement the publisher logic.

SSC Metadata Access Control provides metadata providers with the functionality for specifying access control policies of the SSC metadata documents that they publish. It also assesses and authorises requests for SSC metadata documents. This container was

[6] https://github.com/sigstore/cosign [7] https://github.com/sigstore/rekor

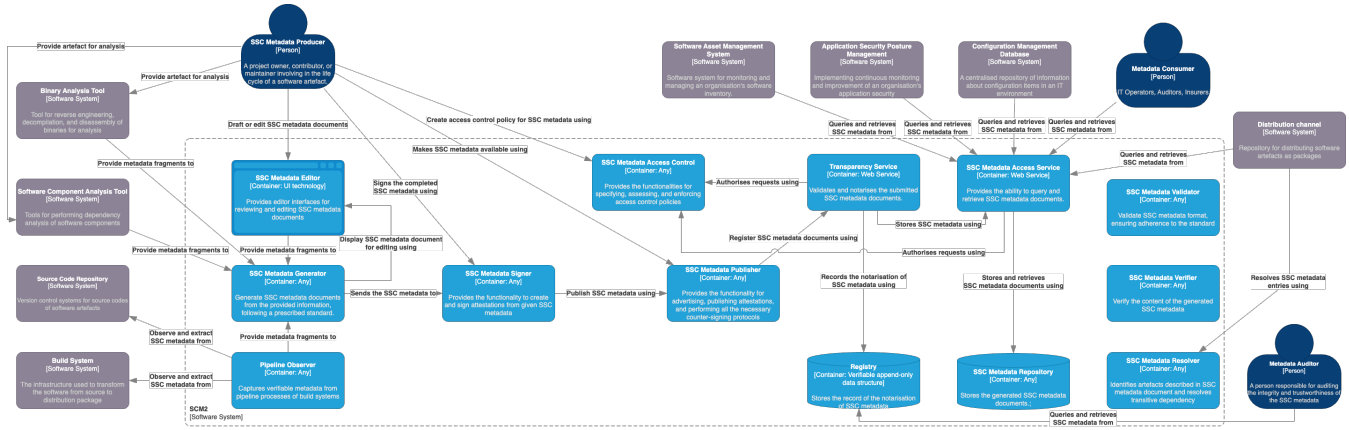

Figure 5: Container model of SCM2

introduced based on the requirement for access control mechanism outlined in the SBOM sharing life cycle [**?** **?**].

SSC metadata sharing group consists of four containers. *SSC Metadata Repository* stores the generated SSC metadata documents. This container corresponds to the "artefact repository" component in the Secure Software Factory reference architecture [**?**]. *SSC Metadata Access Service* acts as a wrapper around the metadata repository, providing the metadata consumers and external systems with the ability to query and retrieve SSC metadata document. This container implements the "access" and "transport" phases of the SBOM sharing life cycle [**?** **?**]. It relies on the SSC metadata access control container to assess and authorise incoming requests.

Registry is a verifiable, append-only data store for the records regarding the notarisation of SSC metadata documents. This container was introduced in the SCITT architecture [**?**]. The public instance of Rekor[8] represents an example of a registry. *Transparency Service* validates and notarises the submitted SSC metadata documents and record the process in the registry. This container was introduced in the SCITT architecture [**?**].

SSC metadata consumption group consists of three containers. *SSC Metadata Validator* provides the ability to validate the syntax of SSC metadata documents to ensure their adherence to a given standard. This component was introduced by the [**?**]. *SSC Metadata Verifier* provides the ability to verify the information contained within a SSC metadata document. This component was introduced by the [**?**]. *SSC Metadata Resolver* implements the resolution step introduced in [**?**]. It identifies artefacts described in SSC metadata document and resolves transitive dependency.

3.3 Variability

This section discusses the variability of SCM2 and presents some prominent variants in terms of the selection, deployment, governance of containers that practitioners can choose when instantiating a concrete SCM2 system for their use case. For brevity, we present variation points and variants in five groups that correspond to architectural design decisions as follows.

Monolithic vs Distributed: This design decision impacts the deployment and interaction between containers making up an SCM2 instance. The result of this decision leads to two SCM2 variants which we denote as *monolithic* and *distributed*. A *monolithic SCM2* operates and scales as a single unit. For instance, the system might contain a core framework that orchestrates SSC metadata life cycle activities and exposes hooks for integration with modules that implement the functionalities of the containers in Figure **??**. The framework and its modules might run within a process or a software container. Scaling of such monolithic SCM2 might be achieved by creating replicas of the system and placing them behind a load balancer.

On the other hand, a *distributed SCM2* comprises multiple units that can operate autonomously and be deployed on separated machines. The containers described in Figure **??** can be mapped directly onto these units. The microservice architecture can be applied to design the architecture of the distributed SCM2 system.

Centralised vs Decentralised: This design decision reflects the relationship between the entities owning and operating a SCM2 system. It influences the security requirements of interaction between SCM2's containers and the need for decentralisation of the SSC metadata registry and repository. A *centralised SCM2* system operates within the trust domain of the system owner. On the other hand, a *decentralised SCM2* system spans multiple trust domains. For instance, the system might have multiple separated SSC metadata producers and SSC metadata providers. In this context, producer's containers cannot trust provider's containers implicitly, and therefore authentication and authorisation mechanisms must be introduced. The SSC metadata registry and repository might need to be replicated and maintained by the consensus of a group of SSC metadata providers (e.g., by leveraging a distributed ledger) to mitigate the lack of trust in a single entity.

Formal metadata distribution vs Informal sharing: This design decision reflects the degree of separation between SSC metadata producers and SSC metadata consumers. When consumers are also producers, such as when an organisation applies a software composition analysis tool to construct SBOM for their software inventory, containers related to the advertisement, access, and transport of SSC metadata can be omitted. We also denote this variant as *single-user SCM2* system. When consumers and providers are separate entities

[8] https://rekor.sigstore.dev

connected via a supply chain relationship, formal distribution and access control mechanisms are necessary. We denote this variant as *multi-user SCM2* system.

Signing: This design decision reflects the maturity of SSC security procedures and SSC metadata adoption within an organisation. For instance, the signing of SSC metadata is only required from level two requirement of both the build track of SLSA [**?**] and the SBOM control family of SCVS [**?**]. We denote the variant of SCM2 without the SSC metadata signer container as *no-sign SCM2* system.

Notarisation: This design decision reflects the maturity of the SSC metadata sharing mechanism. Notarisation includes the countersigning of SSC metadata documents performed by a trusted authority and the recording of such countersigning. For instance, the SCITT architecture [**?**] presents a notarisation protocol. An *unnotarised SCM2* system omits the transparency service and the registry, sending the published SSC metadata documents to the repository directly.

4 EVALUATION

The correctness and utility of the SCM2 RA were evaluated by mapping the SSC security functionality and components of prominent SSC security solutions onto the RA's architectural constructs. It should be noted that since the SCM2 RA was an synthesis of the existing standards and frameworks governing SSC security and SSC metadata, these concrete SSC security solutions and services were not the inputs to the RA and thus can be utilised to test the RA itself. The evaluation aimed at two following research questions:

- **RQ1:** Can the architectural constructs of SCM2 RA describe the functionality and architecture of existing SCM2 solutions?
- **RQ2:** Where do existing solutions focus or deviate from the SCM2 RA?

4.1 Methodology

We selected SSC security solutions for conducting the evaluation by constructing a list of potential solutions from grey literature and filtering these solutions based on whether they implement some SCM2 functionality, described in the context model (Figure **??**). The grey literature articles were gathered via Google search engine using the search phrase "Software Supply Chain Security Solutions OR SBOM Tools." We include articles that list multiple tools and excluded sponsored search results that link to a specific tool. From these articles, we extracted *34 solutions* potential solutions. The solutions were filtered based on their SCM2 support and ranked according to the number of articles citing them. The top five solutions were utilised for the architecture mapping (Table **??**).

4.2 Architecture Mapping

Scribe Platform: Scribe[9] helps software producers and consumers manage risks within their supply chains. It helps them generate evidence such as SBOMs to prove the safety and compliance of their software against regulations and standards such as SLSA. The platform implements the pipeline observer by leveraging the *integration with build systems* like GitHub Actions and Jenkins. Scribe also supports *generating SSC metadata from container images*, such as by leveraging the SBOM generation feature of Docker's BuildKit. Based on inputs from build systems and container image analysis, Scribe can *generate SBOM in CycloneDX format and SLSA-compliant provenance statements*. It also *supports the generation of VeX* to enrich the generated SBOMs. Regarding signing, Scribe supports in-toto attestation by leveraging the `cosign` tool from the `sigstore` toolset. Scribe also supports SSC metadata countersigning by leveraging `Rekor` from the `sigstore` toolset to implement both the registry and transparency service. Regarding SSC metadata storage and distribution, Scribe supports storing SSC metadata in a cloud storage service or a locally deployed OCI-based container registry. A Pub/Sub protocol implements the advertisement and access control of SSC metadata. On the consumer side, Scribe supports verification of the software attestation by leveraging `sigstore` tools.

Chainguard Enforce Platform: The Chainguard Enforce[10] is an SSC security solution focusing on container workloads. It continuously inspects the SSC metadata of container images within a Kubernetes cluster to evaluate and enforce user-defined policies. Whilst Chainguard Enforce focuses on the consumption of SBOM for policy enforcement, it can also *generate SBOM in SPDX and CycloneDX format* for container images that lack an associated SBOM by leveraging Syft[11], an open-source SCA tool for container images. Chainguard Enforce does not extract SSC metadata fragments from build systems. Thus, it *does not include a pipeline observer*. Chainguard Enforce utilises *an OCI-based container registry as the repository* for both containers and the associated SBOM. Producers advertise the existence of SBOM by sending invitations with invite codes. Access is granted via an *Identify and Access Management (IAM) model*. Similar to the Scribe platform, Chainguard Enforce utilises `cosign` and `rekor` from the `sigstore` suite for implementing the signer, registry, and transparency service. `cosign` is also used to verify the signature of the container images.

Anchore Platform: Anchore[12] is an SCM2 focusing on generating and consuming SBOM of container images. Anchore revolves around two open-source tools: Syft for generating SBOM from container images and Grype for consuming SBOM for vulnerability scanning. Anchore does not feature a pipeline observer because it does not capture information about tasks in the build pipeline but triggers the generation of SBOMs and vulnerability analysis upon commits. Anchore utilises PostgreSQL as the data store and supports on-prem databases or integration with external services like Amazon RDS. We could not identify support for SSC metadata signing, notarisation, and signing.

Snyk Platform: Snyk[13] is a developer security platform that provides vulnerability detection across the software life cycle. The platform has been extended to support SBOM generation for external consumption. Snyk can *generate SBOM in SPDX and CycloneDX* format. While the Snyk platform has a GUI, it mainly supports viewing vulnerability reports, whereas viewing the generated SBOMs is carried out by accessing a REST API through the curl command line tool or using the Snyk CLI. We could not identify support for SSC metadata signing, notarisation, and signing.

[9] https://scribesecurity.com/scribe-platform/

[10] https://www.chainguard.dev/chainguard-enforce

[11] https://github.com/anchore/syft

[12] https://anchore.com/opensource/

[13] https://snyk.io/

Table 2: Architectural Mapping of SSC Security Solutions

Container	Scribe	Chainguard	Anchore	Snyk	FOSSA
SSC Metadata Editor	View/ Review and Merge (GUI)	View (GUI)	View (GUI)	View (API, CLI)	View (GUI)
Metadata Generator	SBOM from Container images and Source Code (CycloneDX) VeX related to SBOM SLSA Provenance	SBOM from Container Images (SPDX and CycloneDX) Signed Commits	SBOM from Container Images and Source Code (SPDX and CycloneDX)	SBOM from Container images (SPDX and CycloneDX)	SBOM from source code (SPDX and CycloneDX)
Pipeline Observer	✓	✓	-	-	-
Metadata Signer	Utilizes Sigstore tools which support in-toto-Attestations	Utilizes Sigstore tools which support in-toto-Attestations	Utilizes Syft SBOM tool which supports in-toto-Attestations	-	-
Metadata Publisher	Pub/Sub method	Through invites	-	-	Shareable links
Transparency Service	sigstore Rekor	sigstore Rekor	-	-	-
Metadata Access Control	Pub/Sub method	IAM-based access	-	-	-
Metadata Repository	✓ Cloud Storage, OCI Registry, Local directory	✓ OCI Regsitry	✓ PostgreSQL	-	✓
Registry	Rekor Transparency Log	Rekor Transparency Log	-	-	-
Metadata Access Service	✓	✓	✓	-	✓
SSC Metadata Validator	-	-	-	-	-
SSC Metadata Verifier	✓	✓	-	-	-
SSC Metadata Resolver	✓	✓	✓	✓	✓
Deployment	-	-	Distributed	-	Monolithic
Governance	Centralised	Centralised	Centralised	Centralised	Centralised

FOSSA Platform: FOSSA[14] is an open-source management platform focusing on providing visibility into the licenses and vulnerability of the open source dependencies of a software inventory. FOSSA supports generating SBOM in SPDX and CycloneDX format. It leverages SCA tools to provide fragments to the SBOM generation process. In the SaaS mode of FOSSA, SBOMs are hosted in cloud storage that serves as a repository. Producers can advertise the generated SBOMs by sharing an access link to the repository with consumers. We could not identify support for SSC metadata signing, notarisation, and signing.

4.3 Discussions

The architecture mappings above demonstrated the ability and usability of the SCM2 RA in modelling the functionality and architecture of SCM2 instances. The mappings also reveal some interesting insights about the state of practice of SCM2 and its divergence from academic literature and the existing frameworks.

Containers rather than packages: Three out of five analysed SSC security solutions generate and consume SSC metadata, particularly SBOM, at the software container level. This focus aligns with the cloud native computing paradigm. Nevertheless, it starkly contrasts with the academic literature (e.g., [**?**]) and our experiences in academia-industry collaborative projects, which tend to focus on software packages distributed via package repositories such as NPM and PyPI. An advantage of a container-centric approach is the ease of SSC metadata sharing. The generated SSC metadata documents can be embedded within container images and thus distributed via the same distribution channels. By embedding the SSC metadata inside container images, one digital signature can be used to secure both the artefact (container image) and the SSC metadata document. Furthermore, if a container image carries its corresponding SSC metadata, the resolution step would be simplified as consumers no longer need to link SSC metadata documents with their corresponding software artefacts.

Consumers are producers: Beside Snyk, the analysed SSC security solutions focus on the consumption of SSC metadata for vulnerability detection and policy enforcement. When SSC metadata is generated, it is usually performed in a reverse engineering manner (e.g., analysing an existing container image to extract information and create an SBOM) by the consumers. In other words, the SCM2 implemented by the analysed SSC security solutions belong to the centralised and single-user variant. They exist as utilities of an organisation rather than a diverse ecosystem of SSC metadata producers, providers, and consumers that jointly produce and consume SSC metadata described in the SCITT architecture [**?**] and the SBOM network concept [**?**].

A focus on SBOM: All analysed SSC security solutions focus on SBOM written in SPDX and CycloneDX format rather than software provenance statements in either SLSA [**?**] or in-toto [**?**] provenance model. Even when attestation was utilised, it was used to wrap around SBOM content, such as by using in-toto SPDX predicates[15]. The centralised and single-user design focusing on the consumption of SSC metadata might have contributed to the focus on SBOM rather than provenance.

Attestation and notarisation are not commonplace practices: Signing and notarisation of SSC metadata are not universally supported. Such a state of practice aligns with the requirement levels specified by SLSA [**?**] and SCVS [**?**], which put attestation at a less immediate level than SSC metadata generation. When signing and notarisation were performed, the primary implementation was the `sigstore` software suite.

Reference Instantiation of SCM2: Based on the identified open-source tools, we assembled the following reference instantiation of an SCM2. It covers the end-to-end life cycle of all three types of SSC metadata (SBOM, Provenance and Attestation) from the following SSC layout: software supplier committing the source code to a Version Control System, which is then input into a CI/CD build pipeline to generate a container image as output and publishes it to a Container Image Registry along with SSC metadata related to it. Figure **??** presents the binding of SCM2 containers to open-source tools and technologies.

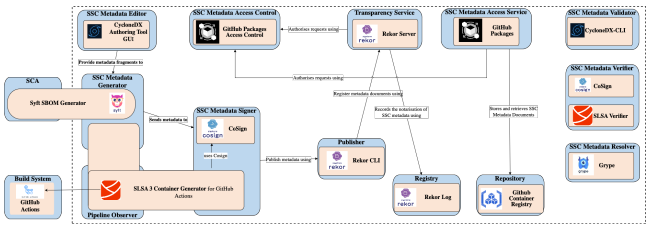

Figure 6: Reference Instantiation of RA

[14] https://fossa.com/

[15] https://github.com/in-toto/attestation/blob/main/spec/predicates/spdx.md

5 CONCLUSIONS

SSC metadata can provide machine-readable and authenticated visibility into the SSC of a software asset inventory, aiding software consumers in protecting themselves against SSC attacks. This paper presented an SoK about SSC metadata in the form of an empirically grounded RA for SCM2. Our RA provides practitioners with a comprehensive domain model and an architectural blueprint for communicating SSC metadata requirements with stakeholders and assembling a suitable SCM2. We demonstrated the correctness and utility of the proposed RA by mapping the architecture of five SSC security solutions with SCM2 onto the RA. The architecture mappings also revealed insights about the state of practice, showing a consumer-driven, SBOM-centric approach to SCM2 and SSC metadata. Because SSC security and SCM2 should be a collaborative effort, for our future work, we plan to develop a software suite based on the RA to facilitate a decentralised and multi-user ecosystem around SCM2 where SSC metadata would be generated from the point of origin, authenticated and notarised by a federation of transparency services and discovered and retrieved by authorised consumers via secure channels. Such a decentralised and multi-user SCM2 ecosystem could serve as a "foundation data layer" on which further security tools, practices, and assurances can be built.

6 DATA AVAILABILITY

Online appendix to this paper is available at https://figshare.com/s/567635a794d7eecf01fe. Appendix contains comprehensive details of the SSC security frameworks used for creating the empirical foundation, knowledge graph constructed from the extracted empirical data and high quality images of all figures used in the manuscript.

7 ACKNOWLEDGEMENT

This work has been supported by the Cyber Security Cooperative Research Centre Limited whose activities are partially funded by the Australian Government's Cooperative Research Centre Program.

Temporary page!

LaTeX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because LaTeX now knows how many pages to expect for this document.